

Broadcast Receiver

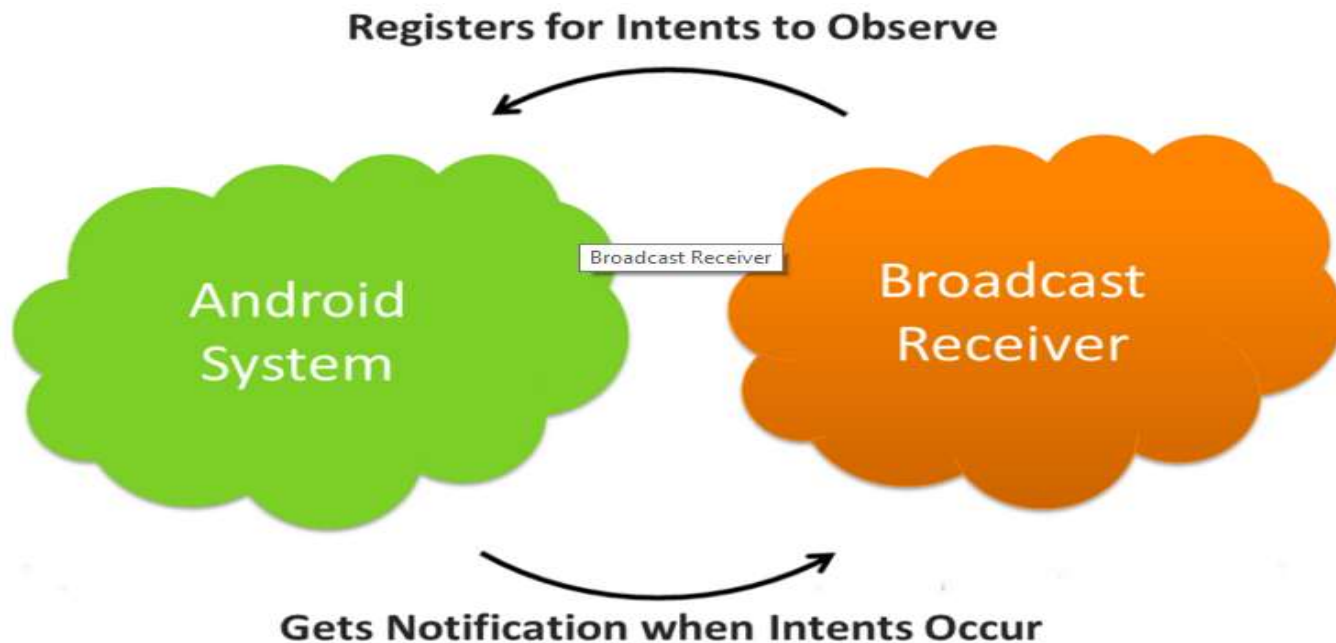
Contents

- **Broadcast**
- **Broadcast receiver implementation**

Broadcast

- **Broadcasts are messages sent by Android system and other Android apps, when an event of interest occurs**
- **Types of broadcast:**
 - System broadcast: are the messages sent by the Android system, when a system event occurs, that might affect your app
 - Custom broadcast: are broadcasts that your app sends out, similar to the Android system.

Broadcast Receiver



Broadcast Receiver

- **Broadcast receivers are app components.**
- **They register for various system broadcast and or custom broadcast.**
- **They are notified (via an Intent):**
 - By the system, when a system event occurs that your app is registered for.
 - By another app, including your own if your app is registered for that custom event.

Implementation

- **Creating the Broadcast Receiver**
 - Extends BroadcastReceiver
 - Implement onReceive method

```
public class MyReceiver extends BroadcastReceiver {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        Toast.makeText(context, "Intent Detected.", Toast.LENGTH_LONG).show();  
    }  
}
```

Implementation

- **Registering Broadcast Receiver**

- Static: Use <receiver> tag in your Manifest file. (AndroidManifest.xml)

```
<application
    android:icon="@drawable/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/AppTheme" >
    <receiver android:name="MyReceiver">

        <intent-filter>
            <action android:name="android.intent.action.BOOT_COMPLETED">
            </action>
        </intent-filter>

    </receiver>
</application>
```

- Dynamic: Use **Context.registerReceiver ()** method to dynamically register an instance.

Register dynamically

- **Register your receiver in onCreate() or onResume().**

// Register the receiver using the activity context.

this.registerReceiver(mReceiver, filter);

- **Unregister in onDestroy() or onPause().**

// Unregister the receiver

this.unregisterReceiver(mReceiver);

Implementation

- Dynamic registration example:

```
MyBroadcastReceiver mReceiver = new MyBroadcastReceiver();

@Override
protected void onResume() {
    super.onResume();

    IntentFilter intentFilter = new IntentFilter();
    registerReceiver(mReceiver, intentFilter);
}

@Override
public boolean onKeyDown(int keyCode, KeyEvent event) {
    if (keyCode == KeyEvent.KEYCODE_MENU) {
        Intent i = new Intent(this, MyIntentService.class);
        startService(i);
    }
    return super.onKeyDown(keyCode, event);
}

@Override
protected void onPause() {
    super.onPause();
    unregisterReceiver(mReceiver);
}
```

Receiving a system broadcast

- Starting from Android 8.0 (API level 26), static receivers can't receive most of the system broadcasts.
- Use a dynamic receiver to register for these broadcasts.
- If you register for the system broadcasts in the manifest, the Android system won't deliver them to your app.

Custom Broadcast

- Sender and receiver must agree on unique name for intent (action name)
- Define in activity and broadcast receiver

```
private static final String ACTION_CUSTOM_BROADCAST =  
    "com.tutorialspoint.CUSTOM_INTENT";
```

Custom Broadcast

- Use `sendBroadcast()`

```
public void broadcastIntent(View view) {  
    Intent intent = new Intent();  
    intent.setAction("com.tutorialspoint.CUSTOM_INTENT");  
    sendBroadcast(intent);  
}
```

- Create a Broadcast Receiver
- Register/Receive broadcast

Send a custom broadcast

Android provides three ways for sending a broadcast:

- **Ordered broadcast.**
- **Normal broadcast.**
- **Local broadcast.**

Ordered Broadcast

- Ordered broadcast is delivered to one receiver at a time.
- To send a ordered broadcast, use the [sendOrderedBroadcast\(\)](#) method.
- Receivers can propagate result to the next receiver or even abort the broadcast.
- Control the broadcast order with [android:priority](#) attribute in the manifest file.
- Receivers with same priority run in arbitrary order.

Normal Broadcast

- Delivered to all the registered receivers at the same time, in an undefined order.
- Most efficient way to send a broadcast.
- Receivers can't propagate the results among themselves, and they can't abort the broadcast.
- The [sendBroadcast\(\)](#) method is used to send a normal broadcast.

Local Broadcast

- Sends broadcasts to receivers within your app.
- No security issues since no interprocess communication.
- To send a local broadcast:
 - To get an instance of LocalBroadcastManager.
 - Call `sendBroadcast()` on the instance.

LocalBroadcastManager.getInstance(this)

.sendBroadcast(customBroadcastIntent);

Classwork

- **Showing message on app when:**
 - Wifi is on/off
 - Bluetooth is on/off

References

- <https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/>